



OBJECT ORIENTED ANALYSIS AND DESIGN

Ch - 13

ACTIVITY DIAGRAM

A study material for the students of GLS University

What is Activity Diagram

Think of activity diagrams as object-oriented flowcharts.

It tells us:

- What activity happens first?
- What happens next?
- Where can the flow branch?
- Which activities happen simultaneously?
- Where do different flows join?
- Which object is produced/used?
- Which person/department is responsible for an activity?

Activity Diagram

They show a process as a set of activities (tasks) connected by transitions (arrows).

Technically, they are a special kind of statechart, where every state is an “activity” that starts running as soon as the state is entered.

Activity = work being performed

Examples:

Login

Select Product

Make Payment

Generate Bill

Deliver Product

Transition = movement from one activity to another

Login → Select Product → Make Payment → Generate Order

Where can you use activity diagrams?

An activity diagram can be attached to any modeling element to model its behavior.

- Use cases (to show the workflow of a use case)
Example - Place Order → Make Payment → Confirm Order
- Classes / interfaces (to show what a class does)
- Components / nodes (to show how they work internally)
- Operations / methods (to show the algorithm for a function)
- Collaborations (To represent the flow involving collaborating elements.)

Action States

Activity diagrams contain:

- **Action states**
- **Subactivity states**

An action state is basic building block of an activity diagram.
Represents a single, atomic task (smallest unit of work).
Used to show “what happens” at a certain step in the workflow.

Properties of Action States:

Atomic → cannot be broken into smaller steps.

Example: Open door

Uninterruptible → once started, it must finish.

You can't “pause” or “interrupt” an action state.

Instantaneous → This does not mean that the real-world action literally takes zero seconds.

It means: From the perspective of the activity workflow, its duration is treated as negligible.

How is it drawn?

Shape: rectangle with rounded ends.

Inside: write the action expression (what the action does).

Action expression can be written as:

- Pseudo-code
- Plain English

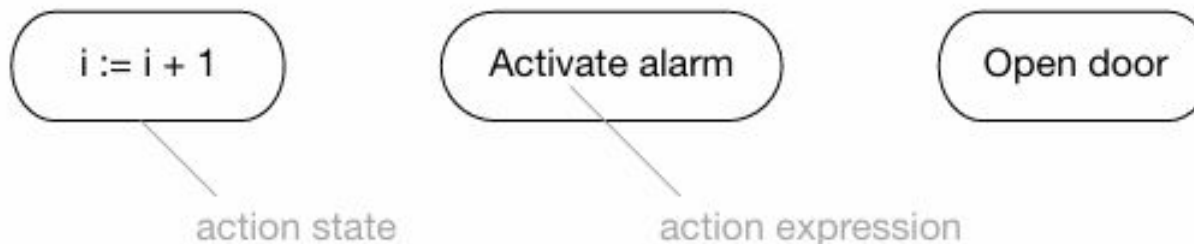
Should be a verb or verb phrase (since actions “do something”).

Why verb?

Because an action means something is being done.

Bad: User, Student, Payment

Better: Validate User, Register Student, Process Payment



Start and Stop States

Start state → solid filled circle ●.

It indicates: Where the activity starts.

Stop state → circle with a ring around it ◎.

It indicates: Where the activity ends.



(●) --> [Write letter] --> [Post letter] --> (◎)

Subactivity States

Action State - One small atomic activity.

A subactivity state is different from an action state.

Subactivity State - A larger activity that can itself be broken into another complete activity diagram.

It is:

- Non-atomic
- Can be broken down into other subactivity and action states.
- Can be interrupted.
- May take a finite amount of time.

An action state: Cannot be broken down.

A subactivity state: Can be broken down into a complete activity graph.

Subactivity States

Example

Suppose we have: Process Order

At a high level, this is one activity.

But internally:

Process Order

```
|  
+-- Validate Order  
|  
+-- Check Stock  
|  
+-- Calculate Price  
|  
+-- Confirm Order
```

Therefore Process Order can be represented as a subactivity state.

Subactivity States

Drawn like an action state (rounded rectangle), but with a thicker border and usually with a small icon (rake-like symbol) at the bottom-right corner.



Action State

Atomic

Cannot be broken down

Uninterruptible

Generally instantaneous

Represents one task

Subactivity State

Non-atomic

Can be broken down

Interruptible

May take finite time

Contains an entire activity graph

Transitions

A transition represents the movement from one state to another in an activity diagram.

When an action or subactivity state completes its work, the diagram moves automatically to the next state.

This kind of movement is called an automatic transition because it does not require a triggering event; it occurs as soon as the current state finishes its activity.

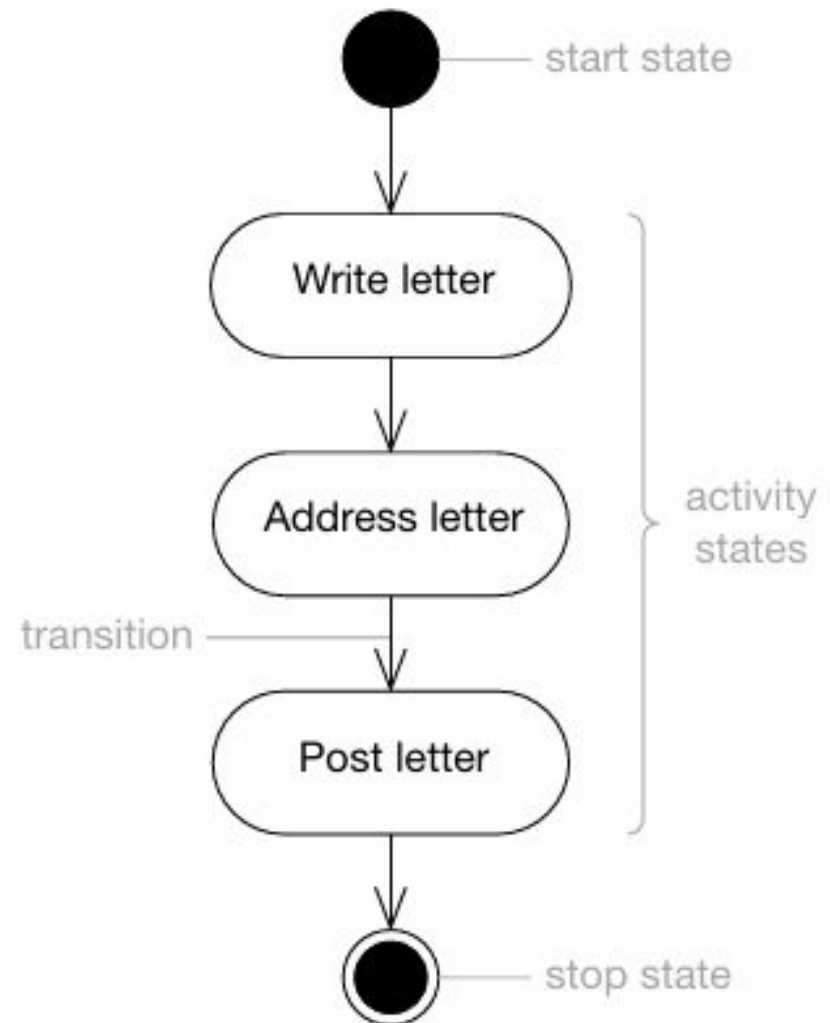
Start State: The activity begins here.

Write Letter (Action State): Automatically transitions from the start state. The system executes this action.

Address Letter (Next Action State): Once "Write Letter" is done, it automatically transitions here.

Post Letter (Next Action State): After addressing the letter, the activity transitions to posting it.

Stop State: After posting the letter, the activity ends here.



Decisions

A decision represents a point where the workflow can branch into alternative paths based on some Boolean condition.

In UML, a decision is shown as a diamond shape, just like in traditional flowcharts.

The same diamond is also used for a merge, where multiple alternative paths come back together.

Key Rules:

One input, many outputs:

Only one path can enter a decision.

Multiple paths may exit, each with a guard expression.

Guard expressions:

Each outgoing path has a Boolean condition (e.g., [is junk]).

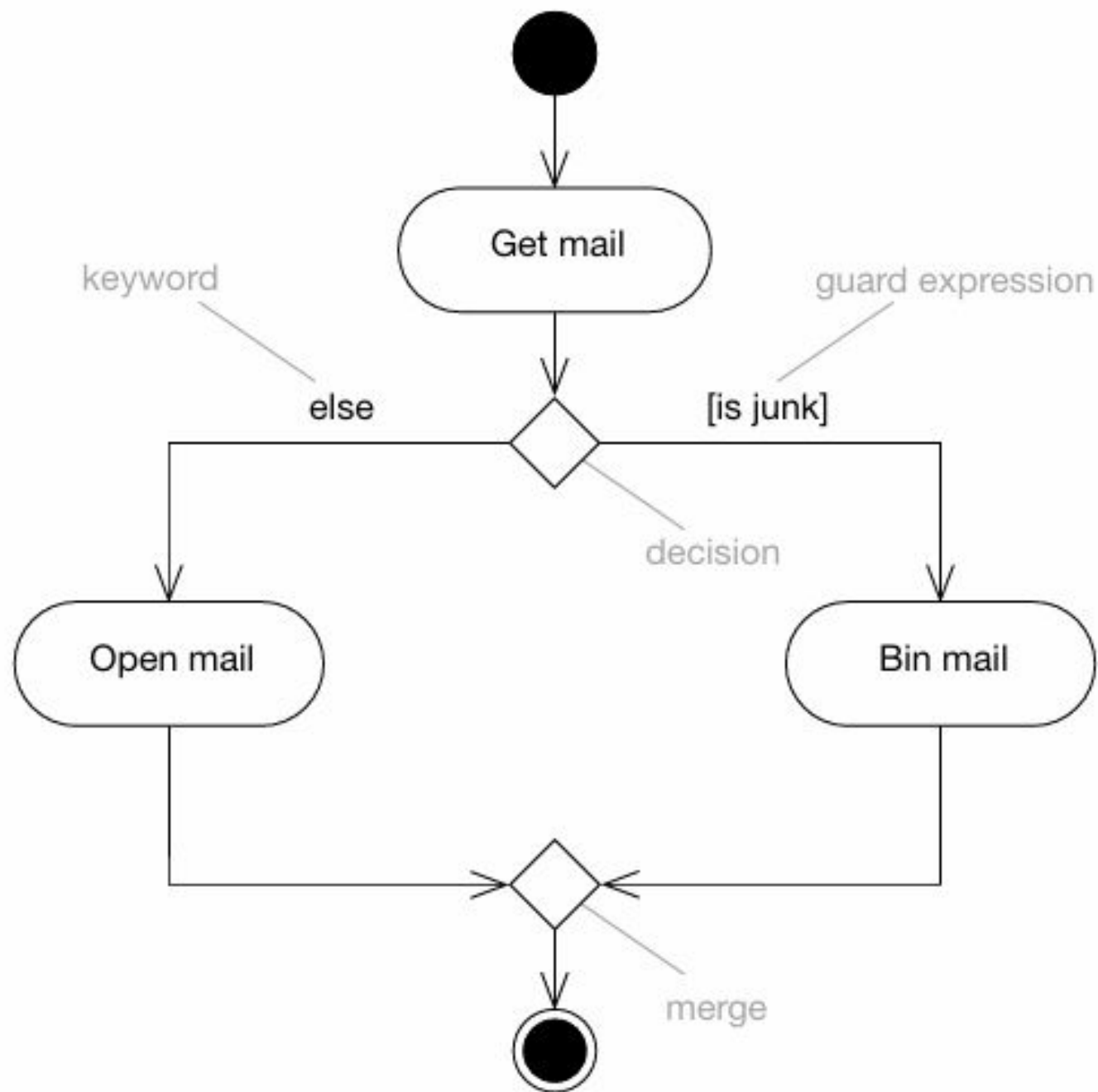
A path is taken only if its guard expression evaluates to true.

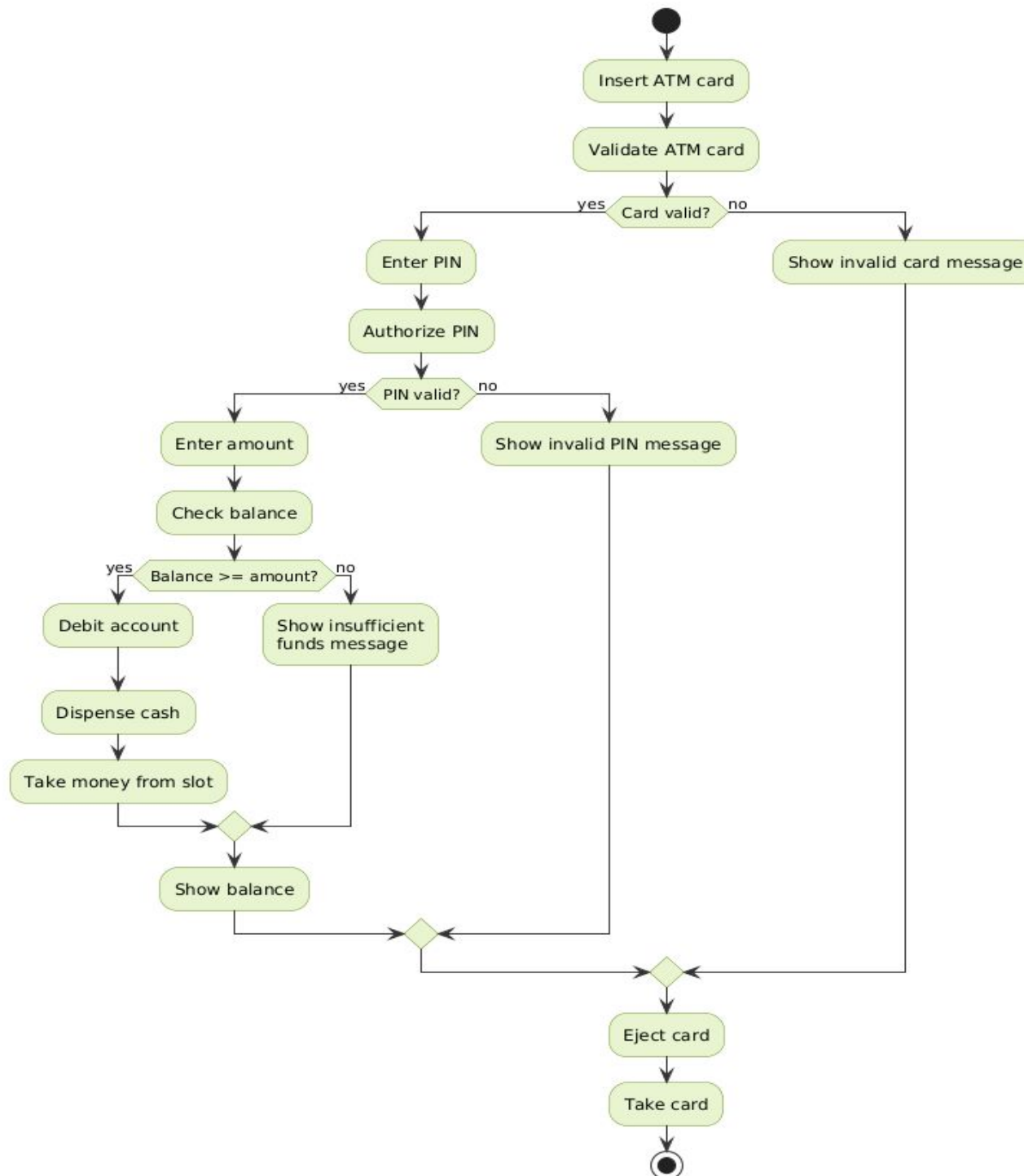
Else path:

The else keyword specifies the path to take if none of the guard expressions are true.

Mutually exclusive guards:

The conditions should not create ambiguity.





Forks and Joins

Fork: Splits a single workflow path into two or more concurrent flows.

Join: Synchronizes two or more concurrent flows back into a single path.

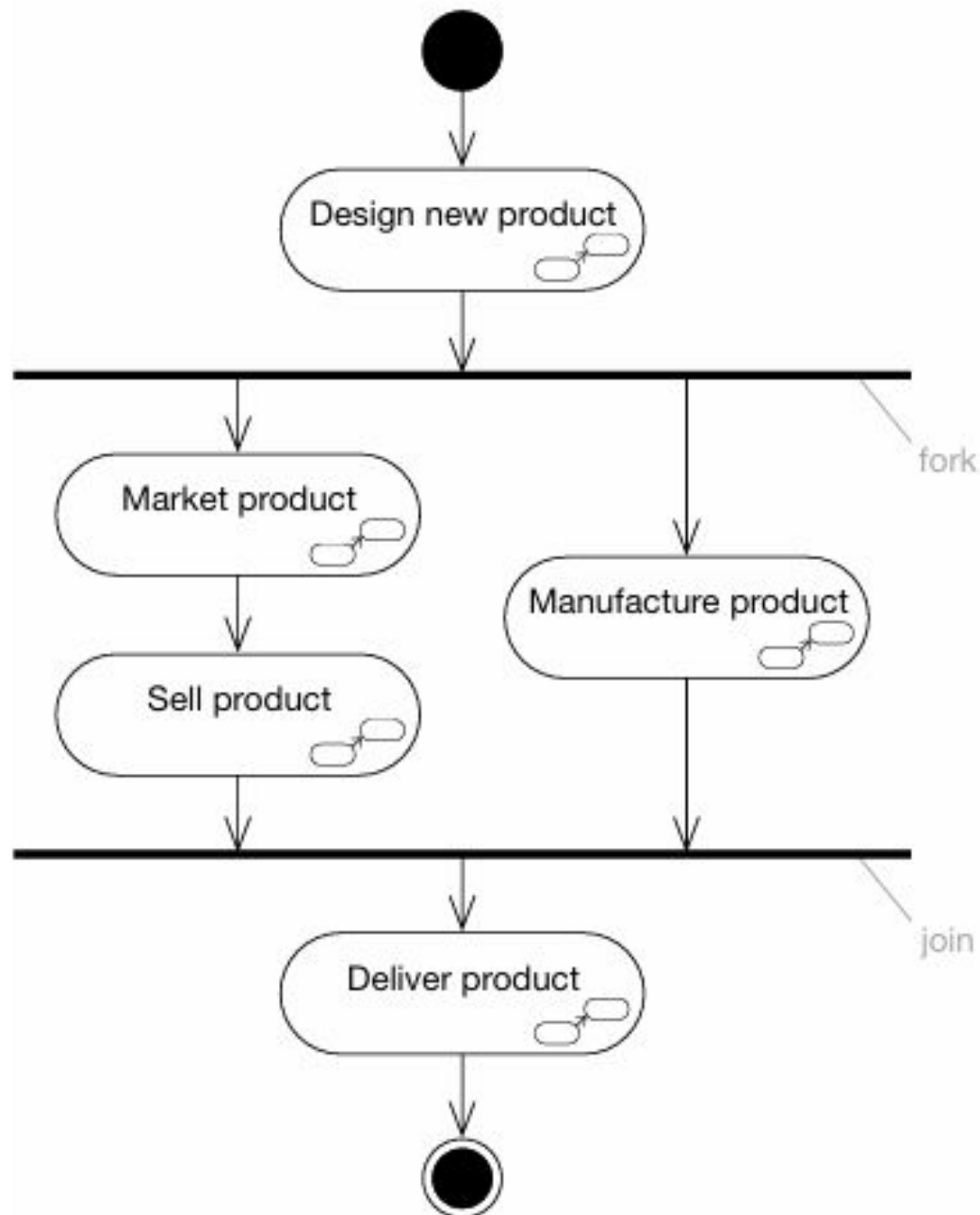
Key Rules:

Forks: One incoming transition.

Two or more outgoing transitions, representing parallel/concurrent activities.

Joins: Two or more incoming transitions.

One outgoing transition, which fires only when all incoming transitions have completed.



Fork vs Join

Fork

Splits a flow

1 incoming transition

2 or more outgoing transitions

Starts concurrent flows

One to Many

Join

Synchronizes flows

2 or more incoming transitions

1 outgoing transition

Waits for concurrent flows to finish

Many to One

Swimlanes

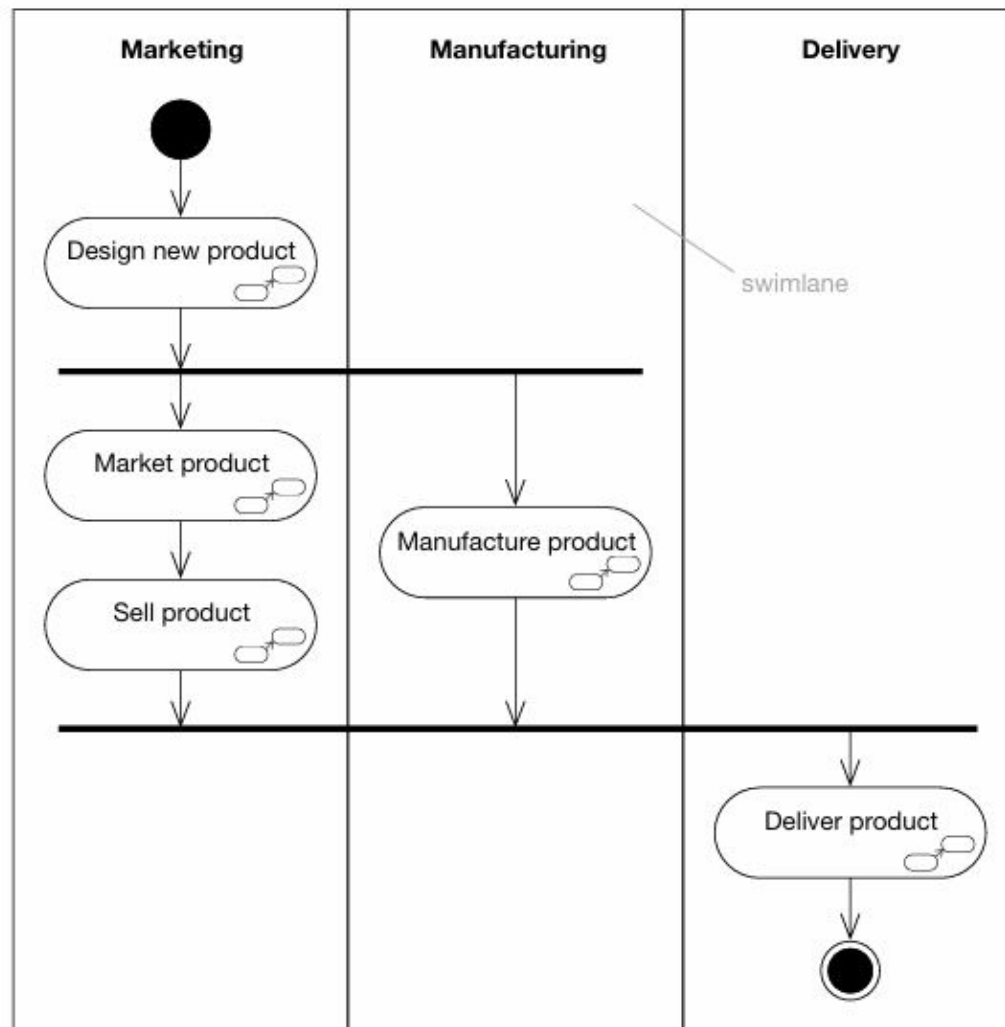
Swimlanes partition an activity diagram into distinct vertical or horizontal sections, called lanes.

UML does not prescribe detailed semantics for swimlanes, so you can use them flexibly.

Common Uses:

Swimlanes are often used to represent:

- Use cases
- Classes
- Components
- Organizational units (e.g., departments in business modeling)
- Roles (e.g., in workflow modeling)



swimlane gives us responsibility information in addition to the workflow.

Without swimlanes: We know what happens.

With swimlanes: We know what happens + who/which role is responsible.

Object Flows

An object flow shows -

- how objects move between activities;
- how objects are inputs to activities;
- how objects are outputs;
- how objects are modified.

What data/object is being passed from one activity to another?

Key Concepts:

Object as Input/Output:

An object can be produced by one activity and consumed by another.

Example: Design new product → :ProductSpec → Manufacture product

So: Design new product produces :ProductSpec.

Then: Manufacture product uses :ProductSpec.

Object State:

The state of an object can be shown using square brackets below the object name.

Example:

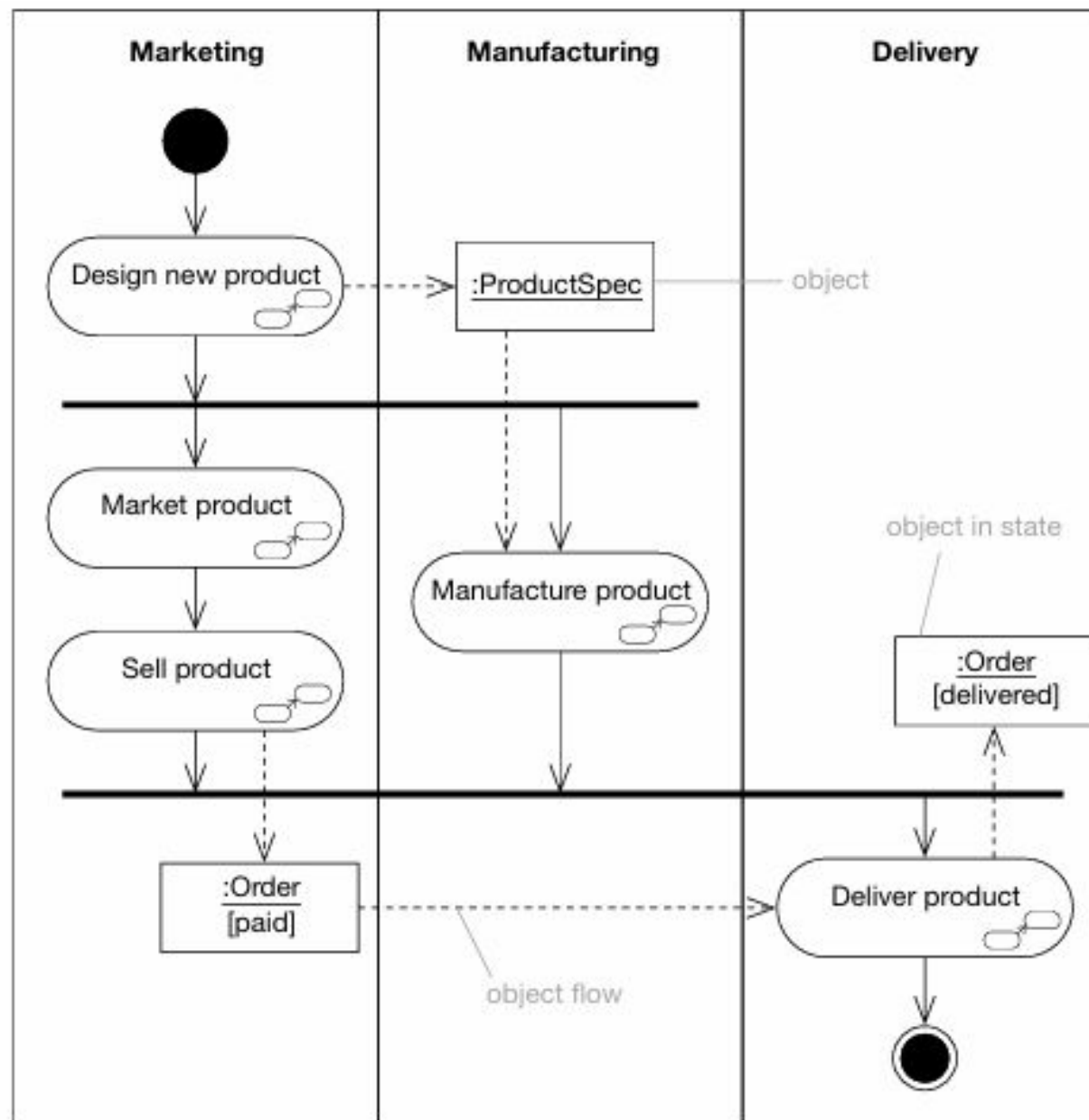
:Order [paid] after payment is captured.

:Order [delivered] after the product is delivered.

The state name should correspond to the object's statechart.

Object Attribute Values:

If the class does not have a statechart, you can show attribute values in a compartment below the object name to indicate its state or changes.



Signals

A Signal represents an event/message that is sent from one part of a system to another, usually without the sender waiting for an immediate response.

Example -

sending a WhatsApp message:

You send the message → the receiver gets notified → you don't have to wait for the receiver to reply before continuing your work.

A signal event occurs when an object receives a signal.

Unlike normal events, signals can be shown on activity diagrams; most other events are shown only on statecharts.

Key Concepts:

Signal Send: Modeled as a convex pentagon labeled with the signal name.

- Has one input and one output transition.
- No actions are performed; it only sends the signal.
- A dashed arrow is drawn from the signal send to the receiving object.

Example: Order signal is sent to :MailOrderCompany.

Payment Completed → Send Signal → "Payment Successful"

Signal Receipt: Modeled as a concave pentagon labeled with the signal name.

- Represents waiting to receive a signal.
- No actions are associated; it merely receives the signal.

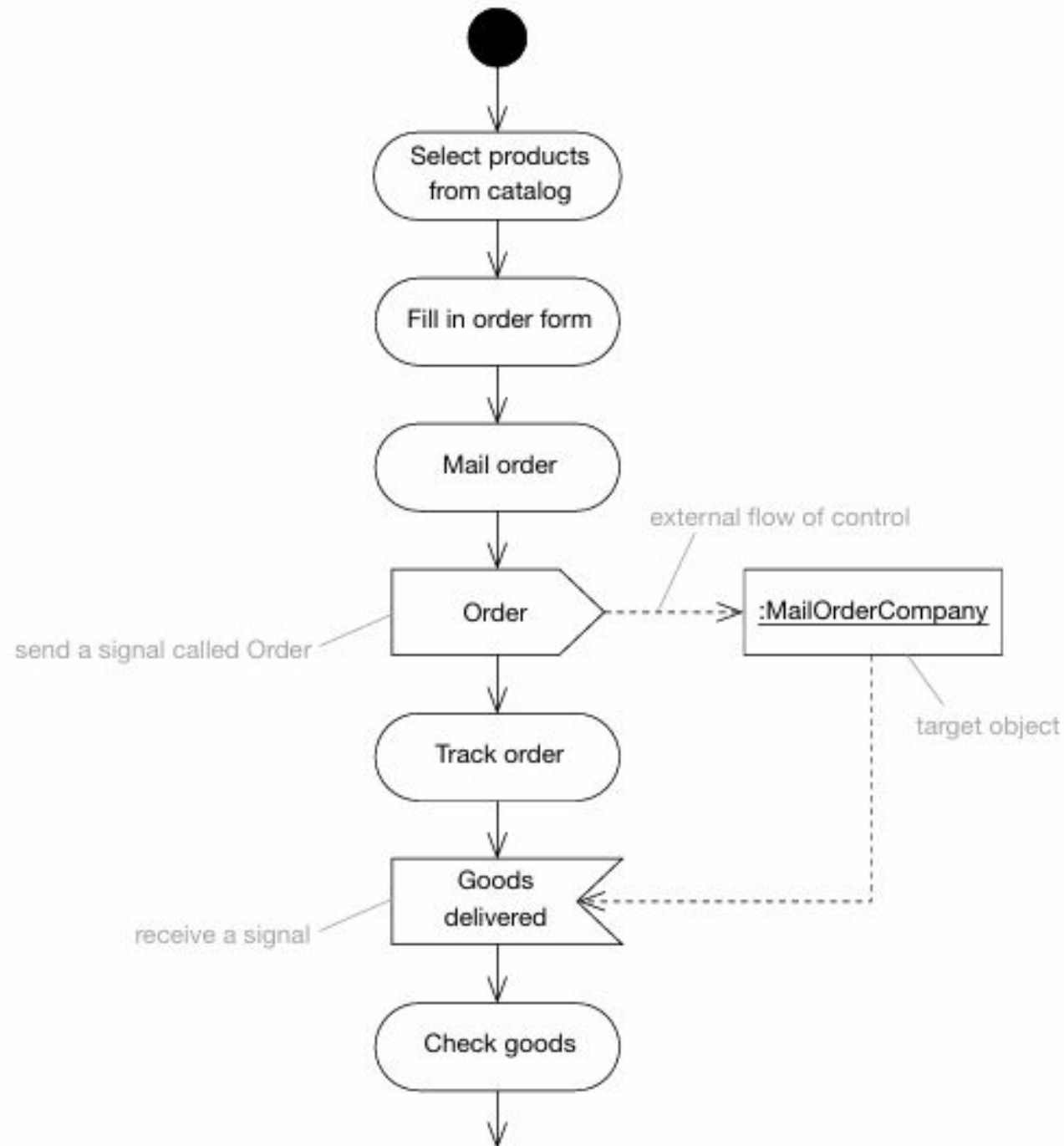
Example: Receiving Goods delivered signal from :MailOrderCompany.

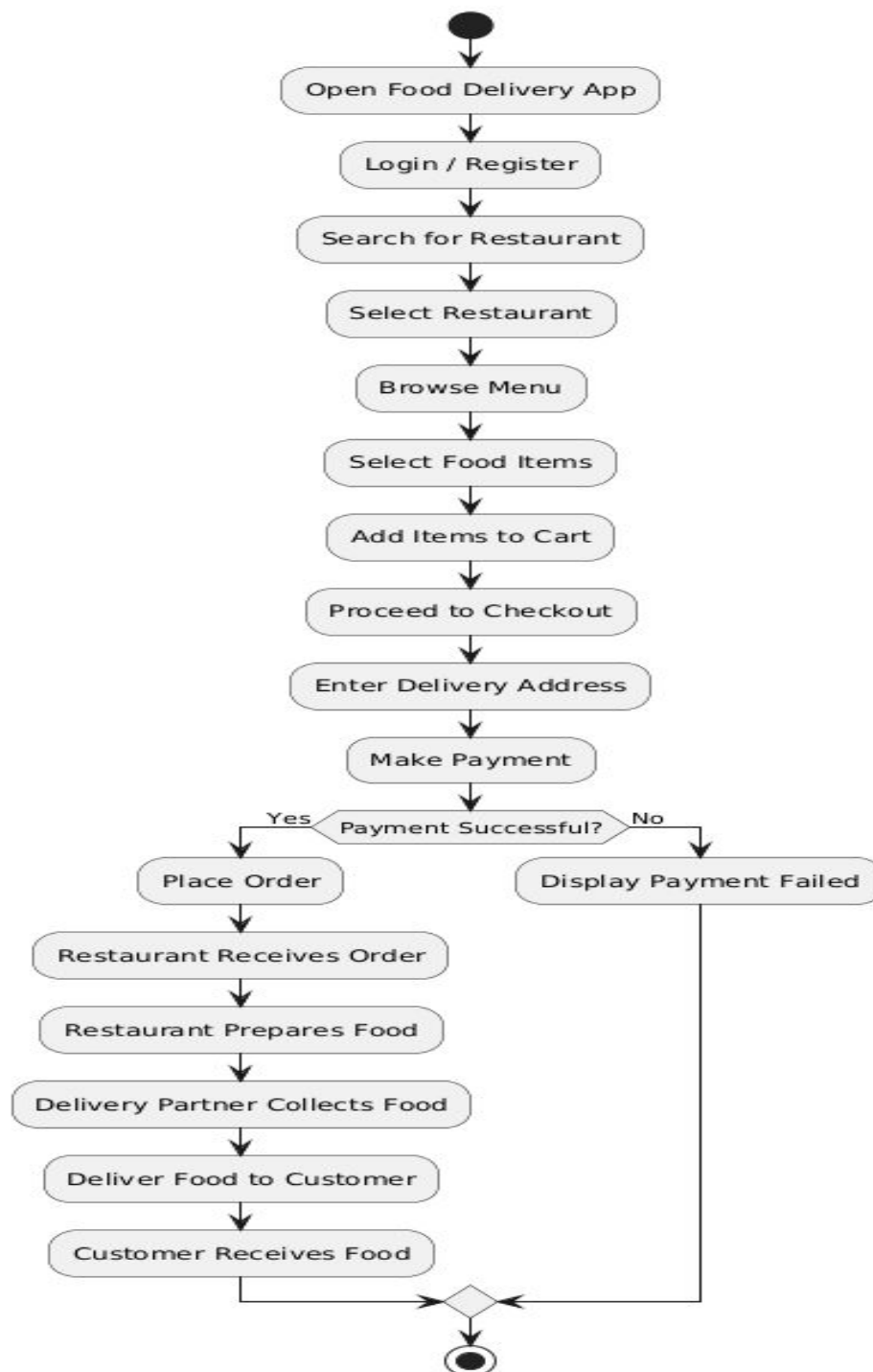
Signal Attributes and Operations:

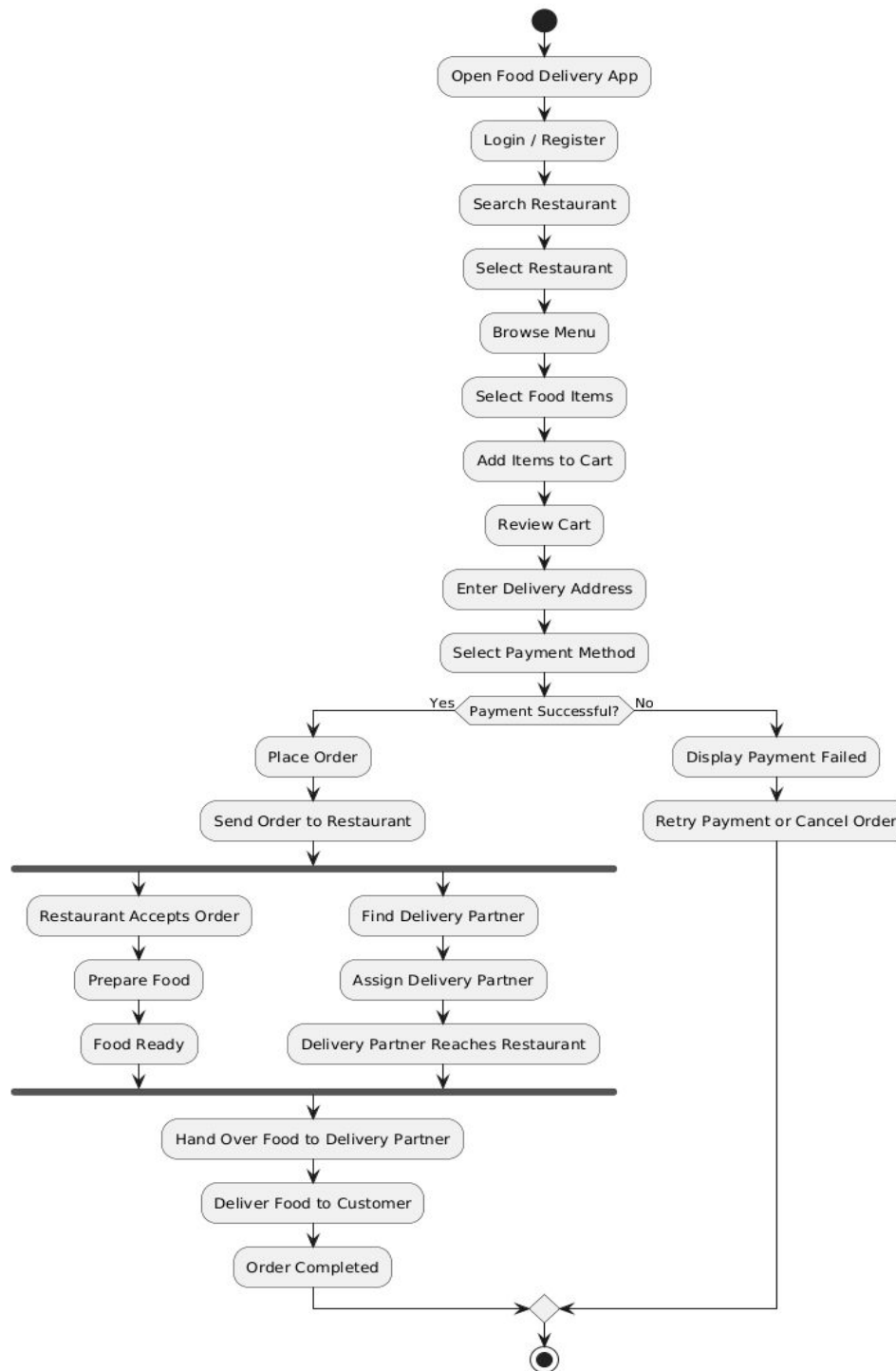
- Signals are classifiers, often modeled as stereotyped classes («signal»).
- They only have attributes representing the information being sent.
- They have an implicit operation send(targetSet), which is not shown in diagrams.
- Signals are primarily for data transfer between objects.

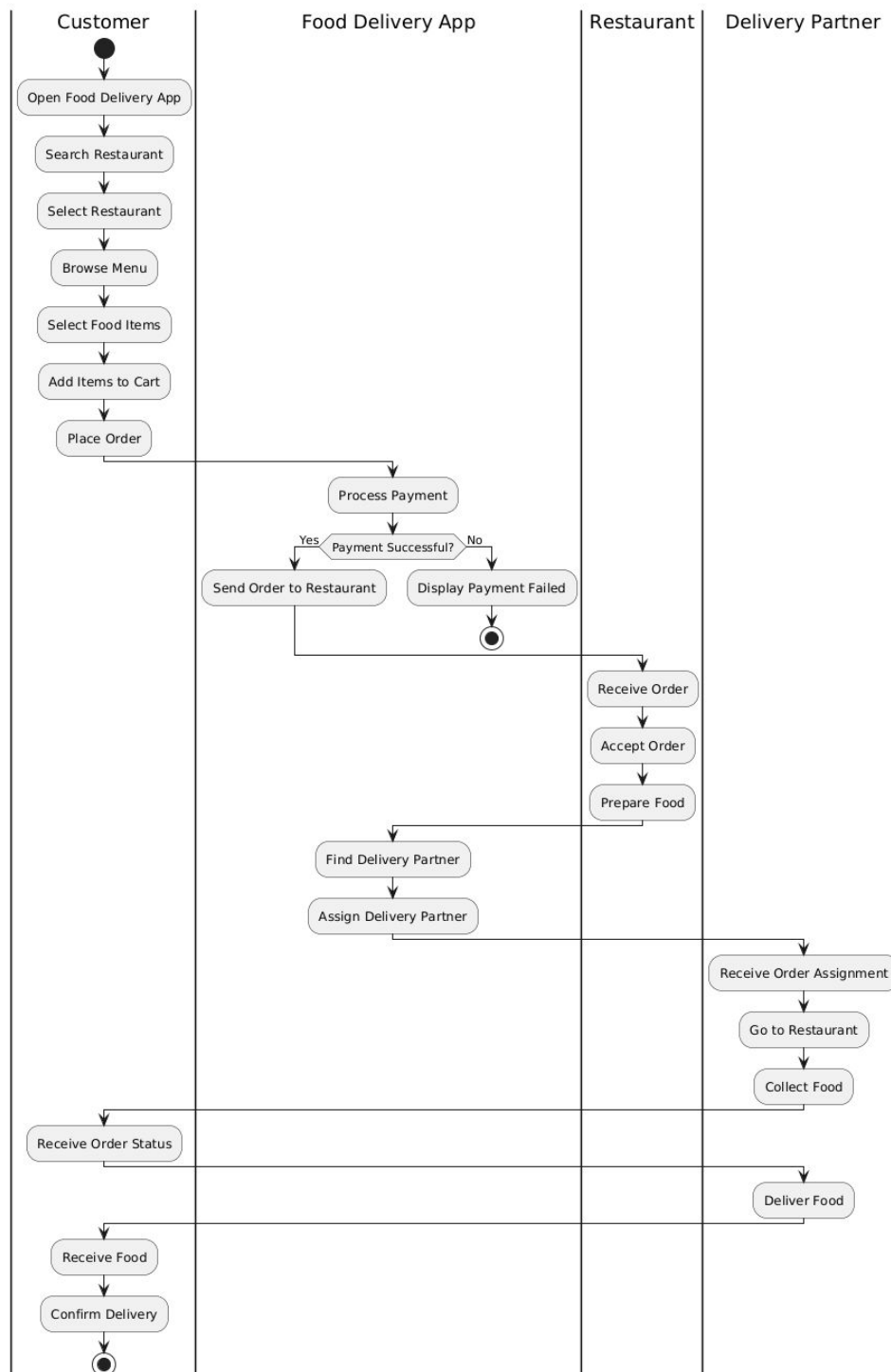
Example – GUI Signals

- «signal» MouseButtonEvent with attributes x, y, button.
- Specialized signals: MouseButtonDown, MouseButtonUp.
- Shows that signals can represent real-world events as data objects.

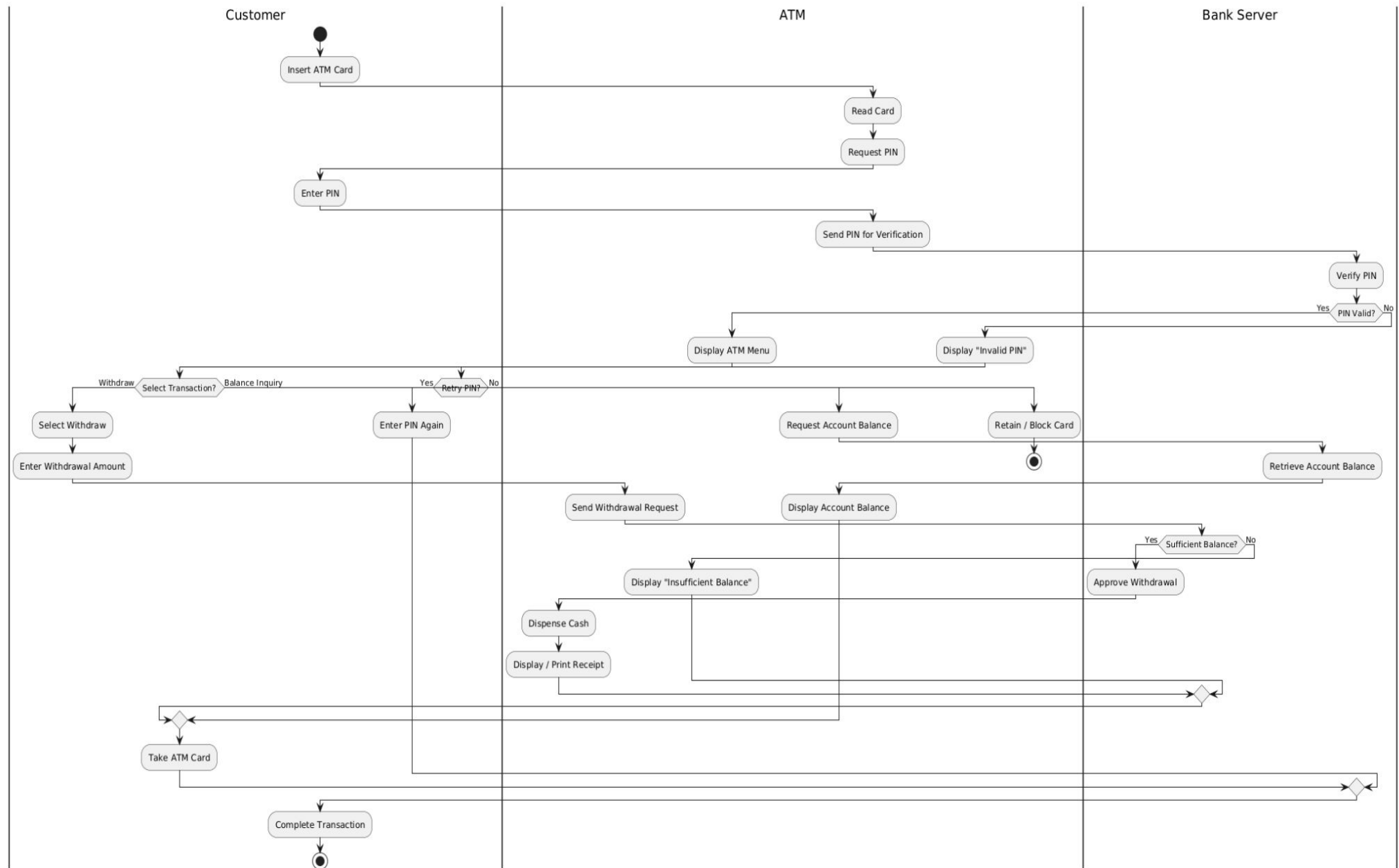








ATM Transaction - Swimlane Activity Diagram



Thank You

GLS FCAIT MSc-(IT)